

```

1 import java.util.ArrayList;
2
3 public class BelmanFord {
4     //First things first, we need an array that maintains the minimum distance as of nwo for every vertex apart fro
5     //Then, we need a 2D array that gives cost from one vertex to another.
6     //Now, how can we structure the 2D array?
7
8     public static void main(String[] args) {
9         belman('A', 6);
10    }
11
12
13    public static ArrayList<Cost> input (){
14        ArrayList<Cost> edges = new ArrayList<>();
15
16        edges.add(new Cost('A', 'B', 6));
17        edges.add(new Cost('A', 'C', 4));
18        edges.add(new Cost('A', 'D', 5));
19        edges.add(new Cost('D', 'C', -2));
20        edges.add(new Cost('C', 'B', -2));
21        edges.add(new Cost('D', 'F', -1));
22        edges.add(new Cost('B', 'E', -1));
23        edges.add(new Cost('C', 'E', 3));
24        edges.add(new Cost('E', 'F', 3));
25
26        return edges;
27    }
28
29    public static ArrayList<Cost> input2 (){
30        ArrayList<Cost> edges = new ArrayList<>();
31
32        edges.add(new Cost('A', 'B', 6));
33        edges.add(new Cost('A', 'C', 4));
34        edges.add(new Cost('A', 'D', 5));
35        edges.add(new Cost('D', 'C', -2));
36        edges.add(new Cost('C', 'B', -2));
37        edges.add(new Cost('D', 'F', -1));
38        edges.add(new Cost('B', 'E', -1));
39        edges.add(new Cost('C', 'E', 3));
40        edges.add(new Cost('E', 'F', 3));
41
42        return edges;
43    }
44    static void belman(char source, int vertices){
45
46        ArrayList<Cost> edges = input();
47
48        //dist[0] = distance from A to B
49        //dist[1] = distance from A to C
50        //...
51        //dist[4] = distance from A to F
52        int[] dist = new int[vertices];
53        for (int i = 0; i < dist.length; i++) {
54            dist[i] = Integer.MAX_VALUE;
55        }
56        dist[source - 'A'] = 0;
57
58        for (int i = 0; i < vertices - 1; i++) {
59            for (Cost edge : edges) {
60                int u = edge.start - 'A';
61                int v = edge.end - 'A';
62                int weight = edge.cost;
63
64                if (dist[u] != Integer.MAX_VALUE && dist[u] + weight < dist[v]) {
65                    dist[v] = dist[u] + weight;
66                }
67            }
68        }
69
70        System.out.println("Vertex Distance from Source");
71        for (int i = 0; i < vertices; i++) {
72            System.out.println((char) (i + 'A') + "\t\t" + dist[i]);
73        }
74    }
75 }
76

```

```
77
78     private static class Cost{
79         char start;
80         char end;
81         int cost;
82
83         Cost(char start,char end,int cost){
84             this.start = start;
85             this.cost = cost;
86             this.end = end;
87         }
88     }
89 }
90
91
92 /*
93 Output:
94
95 Vertex Distance from Source
96 A 0
97 B 1
98 C 3
99 D 5
100 E 0
101 F 3
102
103 */
```